Asymptotic behavior and Big-$\mathcal{O}$ Notation

Clarification of notation

The following statements all mean the same thing:

"$f(x)$ is $\mathcal{O}(g(x))$" OR

"$f(x)$ is of $\mathcal{O}(g(x))$" OR

"$f(x) = \mathcal{O}(g(x))$" OR

"$f(x) \in \mathcal{O}(g(x))$"

$\mathcal{O}(g(x))$ is a collection of functions (i.e. a set) so what we should say is $f(x) \in \mathcal{O}(g(x))$, but $f(x) = \mathcal{O}(g(x))$" is commonly used. This can be confusing since $\mathcal{O}(n) = \mathcal{O}(n^2)$ but $\mathcal{O}(n^2) \neq \mathcal{O}(n)$!!

See this interesting thread on StackExchange Big O Notation "is element of" or "is equal". Note the Wiki cites Donald Knuth.

Big-$\mathcal{O}$ classification of common functions

Order of Asymptotic Behaviour. Remember that these are sets. So while it might be said that $2n + 3$ "is" $\mathcal{O}(n)$; actually it's an element in that set: $2n + 3 \in \mathcal{O}(n) \subset \mathcal{O}(n^2) \subset \ldots$.

Here's a list of some functions listed from fast to slow.

$\mathcal{O}(1)$ **Constant**

$\mathcal{O}(\log(n))$ **Logarithmic**.

Ignore Log bases: $\log_{10}(n) = \log_2(n) = \log(n)$

Same growth as $\log(n^k) = k \log(n)$

$\mathcal{O}(\log^c(2n))$ **Polylogarithmic**

Don't forget that: $\log^c x = (\log x)^c$.

$\mathcal{O}(n)$ **Linear**

$\mathcal{O}(n \log(2n))$ **Linearithmic/Loglinear**

$\mathcal{O}(n^2)$ **Quadratic**

$\mathcal{O}(n^2 \log(2n))$

$\mathcal{O}(n^3)$ **Polynomial**

$\mathcal{O}(2^n)$ **Exponential**

$\mathcal{O}(n!)$

Big-$\mathcal{O}$ problems

1 Order each function according to their growth from fastest to slowest.

(a) $n \cdot \log 2n$

(b) $\log_{10} n$

(c) 10^n

(d) n^{10}

(e) $10n$

(f) 100

(g) n^2

Solution:

(f) 100

(b) $\log_{10} n$

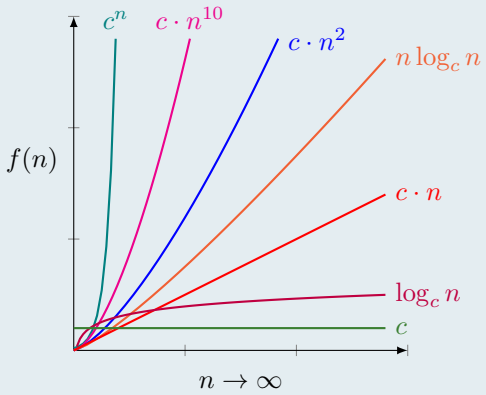
(e) $10n$

(a) $n \cdot \log 2n$

(g) n^2

(d) n^{10}

(c) 10^n



2 The provided expressions are the processing time of an algorithm for problems of size n. For each expression, find the *lowest possible* Big-$\mathcal{O}$ complexity stated in simplest form.

Solution: Given two real functions $f(x)$ and $g(x)$ such that $g(x)$ is strictly positive for sufficiently large x, we say that

$$f(x) \in \mathcal{O}(g(X))$$

if and only if there exist $M, x_0 \in \mathbb{R}$ such that

$$|f(x)| \leq M \cdot g(x)$$

for all $x_0 \leq x$ This formal definition of Big-$\mathcal{O}$ notation is not included in the text. We don't use it directly. Just remember that $f(x)$ is $\mathcal{O}(g(x))$ bound from above by $M \cdot g(x)$, i.e., $\mathcal{O}(g(x))$ is an upper bound.

(a) $100n^2 + 0.0002n^3 + 10,000$

Solution: From the definition above, we can see that coefficients are ignored. As $n \to \infty$, they have little effect. So we need only compare n^2 and n^3. Since $n^3 = n \cdot n^2$, we have $\mathcal{O}(n^2) \subset \mathcal{O}(n^3)$ and our answer is:

$$\mathcal{O}(n^3)$$

(b) $20n + 2n \log_{10} n + 30$

Solution: $\mathcal{O}(n \log_2 n)$ To find this we use the following property:

$$\text{If } f \in \mathcal{O}(f') \text{and } g \in \mathcal{O}(g') \text{ then } f \cdot g \in \mathcal{O}(f \cdot g)$$

So for the product of two functions we can evaluate the factors.
$2n \in \mathcal{O}(n)$ and $\log_{10} n \in \mathcal{O}(\log_2 n)$. So then our answer is:

$$\mathcal{O}(n \log_n)$$

As a matter of convention, the base is often excluded, however $\mathcal{O}(\log_2 n)$ is sometimes written.

Why do we ignore the base in logs but not exponents? Recall that

$$\log_2 n = \frac{\log_{10} n}{\log_{10} 2}$$

So then

$$\log_{10} n \leq (\log_{10} 2) \log_2 n$$

for all n. Here our M is $\log_{10} 2$. The situation is different for the bases of *exponential* functions, but remember that logs are the inverses of exponential functions.

(c) $\log_{10} 5 + \log_2 7$

Solution: Both terms are both constants. So the answer is:

$$\mathcal{O}(1)$$

(d) $5n + \sqrt{n}$

Solution: $\sqrt{n} = n^{1/2}$ and $5n^1 = 5n^{1/2}n^{1/2}$. Now we can see that $\mathcal{O}(\sqrt{n}) \subset \mathcal{O}(n)$. So the answer is:

$$\mathcal{O}(n)$$

(e) $0.01(2^n) + n^5 + \ln n$

Solution: $\mathcal{O}(\log n) \subset \mathcal{O}(n^a) \subset \mathcal{O}(b^n)$ for $a, b \in \mathbb{R}$ and $b > 1$. That is, an exponential function is slower than a polynomial which is slower than a log. So the answer is

$$\mathcal{O}(2^n)$$

To see that $\mathcal{O}(n^a) \subset \mathcal{O}(b^n)$, note that $\log x < \log y \iff x < y$. So then,

$$\log n^a = a \log n < n \log b = \log b^n$$

for sufficiently large n as we know that $\mathcal{O}(\log n) \subset \mathcal{O}(n)$

(f) $100 \cdot 4^n + 200 \cdot 3^n$

Solution: The answer is:

$$\mathcal{O}(4^n)$$

The text incorrectly states that *"the base of the exponent is irrelevant"* in lesson 1.13. $2^n \leq 3^n$ for

all n, but there is no constant M such that $3^n \leq M \cdot 2^n$ for all n. If there was then there would be an integer $x > 0$ such that $M < 3^x$, and then $3^x \cdot 3^{n-x} \leq 3^x \cdot 2^{n-x}$ for sufficiently large n. Which is a contradiction.

3 $f(x) = 6x \log_4 x$ is which of the following? **Mark all that apply.**

◯ $\mathcal{O}(6)$ ◯ $\mathcal{O}(\log x)$ ◯ $\mathcal{O}(6 \log x)$ √ $\mathcal{O}(x \log x)$ √ $\mathcal{O}(x^2)$ √ $\mathcal{O}(2^x)$

Solution: Big-$\mathcal{O}$ is an *upper bound*. So if $f(x)$ is no slower than $x \log x$ then it is also no slower than x^2 and 2^x. In set notation,

$$f(x) \in \mathcal{O}(x \log x) \subset \mathcal{O}(x^2) \subset \mathcal{O}(2^x)$$

4 Let $h(x) = 6x^4 + 2x^3 + x + 100$. Which of the following is true? **Mark all that apply.**

√ $h(x) \in \mathcal{O}(x^4)$ √ $h(x) \in \Theta(x^4)$ √ $h(x) \in \Omega(x^4)$

Solution:

$\mathcal{O}(x^4)$ is an *upper bound*. $\Omega(x^4)$ is a *lower bound*. So then $h(x)$ is also $\Theta(x^4)$, a *tight bound*.

5 Let $h(x) = 6 \log_2 x$. Which of the following is true? **Mark all that apply.**

√ $h(x) \in \mathcal{O}(4x^2)$ ◯ $h(x) \in \Theta(4x^2)$ ◯ $h(x) \in \Omega(4x^2)$

Solution: $6 \log_2 x$ is faster than $4x^2$. So then $h(x) \in \mathcal{O}(4x^2)$ and $h(x) \notin \Omega(4x^2)$. Which means $h(x) \notin \Theta(4x^2$

6 Let $f(x) = x^2 \log_3 x$. Which of the following is true? **Mark all that apply.**

◯ $f(x) \in \mathcal{O}(2x \log_{10} x)$ ◯ $f(x) \in \Theta(2x \log_{10} x)$ √ $f(x) \in \Omega(2x \log_{10} x)$

Solution: Recall that the base of a logarithmic function can be ignored. So $\log_{10} x$ and $\log_3 x$) have the same asymptotic behavior, and we really only need to evaluate x^2 and $2x$. We know that $x^2 \notin \mathcal{O}(2x)$, and so neither can it be in $\Theta(2x)$ which gives us our answer.

7 Let $g(x) = 2x \log_{10} x$. Which of the following is true? **Mark all that apply.**

√ $g(x) \in \mathcal{O}(x^2 \log_3 x)$ ◯ $g(x) \in \Theta(x^2 \log_3 x)$ ◯ $g(x) \in \Omega(x^2 \log_3 x)$

Solution: This is similar to question 6, only here $g(x)$ has a *faster* factor, $2x$. So then $g(x) \in \mathcal{O}(x^2 \log_3 x)$ (it's faster) and $g(x) \notin \Omega(x^2 \log_3 x) \Rightarrow g(x) \in \Theta(x^2 \log_3 x)$.

8 Using the provided pseudo-code, find the worst case performance in Big-$\mathcal{O}$ notation.

Algorithm 1 Some messy pseudo-code

```
1: procedure SomeProcedure
2:     for i=1 and 1<=n do
3:         j=1
4:         while j<n do
5:             j=j+2
```

A. $\mathcal{O}(\log n)$

B. $\mathcal{O}(n \log n)$

C. $\mathcal{O}(n^2)$

D. $\mathcal{O}(n)$

9 Using the provided pseudo-code, find the worst case performance in Big-$\mathcal{O}$ notation.

Algorithm 2 Some more messy pseudocode

```
1: procedure SomeProcedure2
2:     j=0
3:     for i=o; i<n; i++ do
4:         j=i+j
```

A. $\mathcal{O}(\log n)$

B. $\mathcal{O}(n \log n)$

C. $\mathcal{O}(n^2)$

D. $\mathcal{O}(n)$

10 Using the provided pseudo-code, find the worst case performance in Big-$\mathcal{O}$ notation. Assume we know that **someMethod(n)** is $\mathcal{O}(\log n)$.

Algorithm 3 Some pseudocode with a method in it

```
1: procedure SomeProcedure3
2:     j=0
3:     for i=o; i<n; i++ do
4:         j=someMethod(n)
```

A. $\mathcal{O}(\log n)$

B. $\mathcal{O}(n \log n)$

C. $\mathcal{O}(n^2)$

D. $\mathcal{O}(n)$

11 Using the provided pseudo-code, find the worst case performance in Big-$\mathcal{O}$ notation.

Algorithm 4 Some more messy pseudocode

```
1: procedure SomeProcedure4
2:     while n> 1 do
3:         n=n/2
```

A. $\mathcal{O}(\log n)$

B. $\mathcal{O}(n \log n)$

C. $\mathcal{O}(n^2)$

D. $\mathcal{O}(n)$

12 Using the provided pseudo-code, find the worst case performance in Big-$\mathcal{O}$ notation.

Algorithm 5 Euclid's algorithm

```
1: procedure Euclid(a, b)                          ▷ The g.c.d. of a and b
2:     r ← a mod b
3:     while r ≠ 0 do                              ▷ We have the answer if r is 0
4:         a ← b
5:         b ← r
6:         r ← a mod b
7:     return b                                                ▷ The gcd is b
```

A. $\mathcal{O}(\log n)$

B. $\mathcal{O}(n \log n)$

C. $\mathcal{O}(n^2)$

D. $\mathcal{O}(n)$